

Classes

Dr. Sanjit Kumar Saha



Gisma
University
of Applied
Sciences



Object Oriented Programming

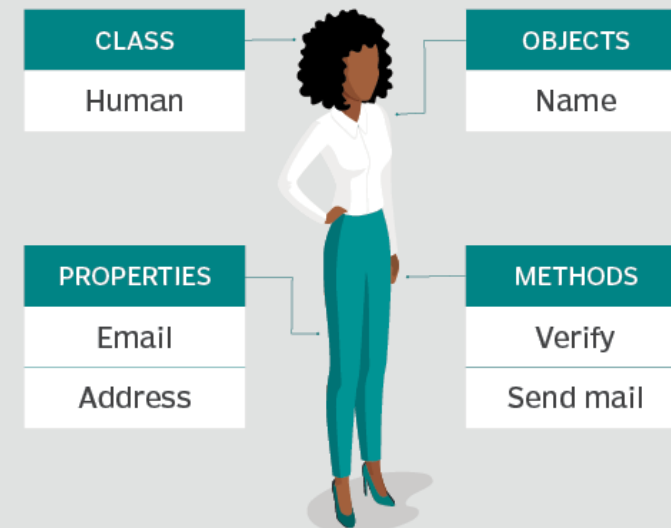
[Hunt 2019]

<https://www.techtarget.com/searcharchitecture/definition/object-oriented-programming-OOP>

2

- Python is an object-oriented language
 - allow to structure code using classes and objects for better organization and reusability
- Advantages of OOP
 - Provides a clear structure to programs
 - Makes code easier to maintain, reuse, and debug
 - Helps keep your code DRY (Don't Repeat Yourself)
 - Allows you to build reusable applications with less code

Object-oriented programming





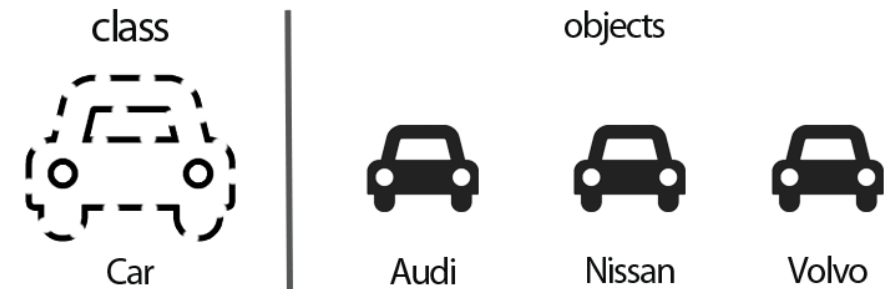
Class

[Hunt 2019]

<https://javatutorial.net/java-oop>

3

- A *class* is a collection of objects
- Classes are blueprints for creating objects
- A class defines a set of attributes and methods that the created objects (instances) can have
 - Classes are created by keyword *class*
 - Attributes are the variables that belong to a class
 - Attributes are always public and can be accessed using the dot (.) operator. Example: *Myclass.Myattribute*



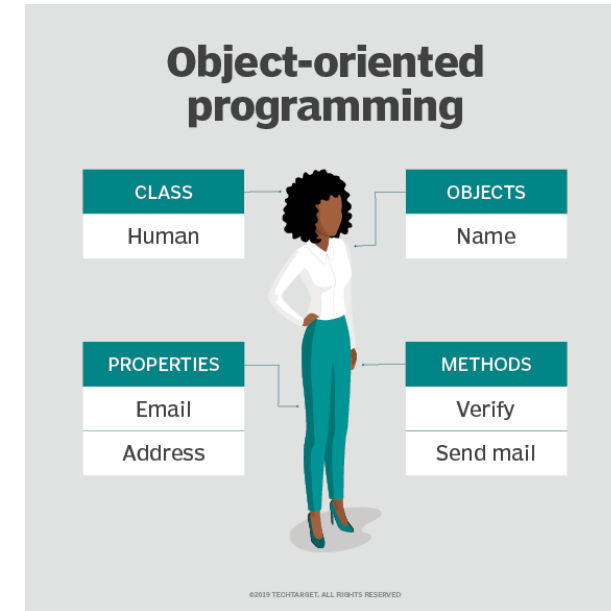


Class Terminology

[Hunt 2019]

4

- **Class**
 - A class defines a combination of data and behaviour that operates on that data
 - A class acts as a template when creating new instances
- **Instance/object**
 - An instance also known as an object is an example of a class
 - All instances of a class possess the same data fields/attributes but contain their own data values
 - Each instance of a class responds to the same set of requests
- **Attribute/field/instance variable/property**
 - The data held by an object is represented by its attributes
 - The “state” of an object at any particular moment relates to the current values held by its attributes
- **Method**
 - A method is a procedure defined within an object
 - It is the name given to behaviour that is linked directly to the class
 - It is not a free-standing function rather it is part of the definition of the class





Creating a *Class*

- *class* keyword indicates that we are creating a class followed by name of the class

class **Person**:

species = "**Human**" # *Class attribute*



- *class* **Person**: creates a class named Person, which acts as a blueprint for person objects
- *species* is a class attribute, meaning it is shared by all instances of the class



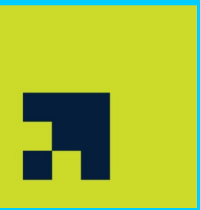
What Are Classes for?

- A class should accomplish one specific purpose
 - It should capture only one idea
- Classes are used to represent real-world entities
 - Such as employees
- They can also represent more abstract concepts
 - Such as a transaction between one person and another



Objects

- An *object* is an instance of a Class.
- An object represents a specific implementation of the class and holds its own data.
- An *object* consists of:
 - *State*: It is represented by the attributes and reflects the properties of an object.
 - *Behavior*: It is represented by the methods of an object and reflects the response of an object to other objects.
 - *Identity*: It gives a unique name to an object and enables one object to interact with other objects.



Creating an *Object*

- Creating an *object* involves instantiating a class to create a new instance of that class.
- This process is also referred to as object instantiation.

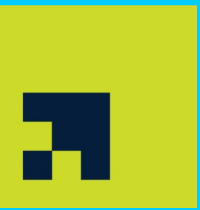
```
class Person:
```

```
    species = "Human" # Class attribute
```

```
# Creating an object of the Dog class
```

```
person1 = Person()
```

```
print(person1.species)
```

Multiple Objects

- We can create multiple objects from the same class:

```
class Person:  
    species = "Human" # Class attribute
```

```
# Creating an object of the Dog class
```

```
person1 = Person()  
person2 = Person()  
person3 = Person()
```

```
print(person1.species)  
print(person2.species)  
print(person3.species)
```

- Each object is independent and has its own copy of the class properties

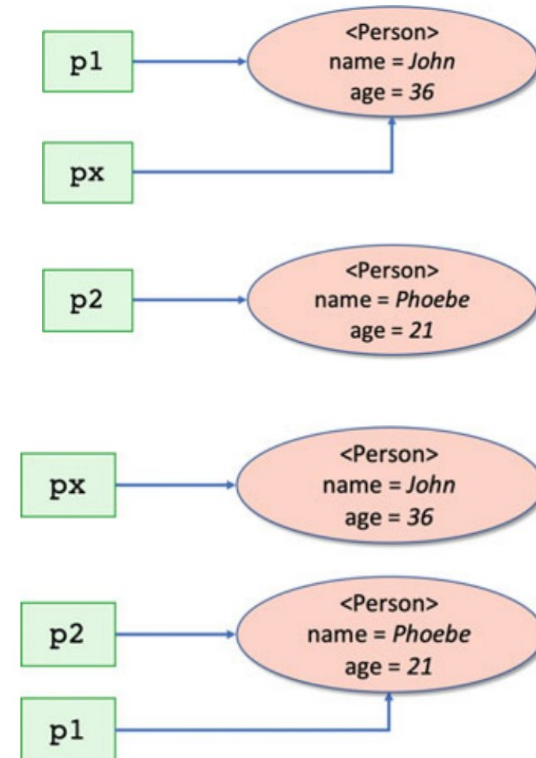


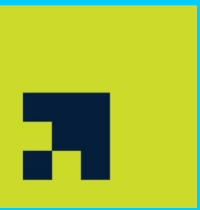
Be Careful with Assignment

[Hunt 2019]

10

- $p1$ and $p2$ reference different instances of the class Person
 - What happens when $p1$ or $p2$ are assigned to another variable?
- $px = p1$
 - It makes a complete copy of the value held by $p1$
 - However, $p1$ does not hold the instance of the class Person
 - It holds the address of the object
 - Both $p1$ and px now reference (point at) the same instance in memory
- $p1 = p2$
 - If $p1$ is subsequently assigned a different object, then this would have no effect on the value held in px





The *pass* Statement

- Class definitions cannot be empty
 - if we for some reason have a class definition with no content, put in the `pass` statement to avoid getting an error

```
class Person:  
    pass
```



The `__init__()` Method

[Hunt 2019]

12

- All classes have a built-in method called `__init__()`, which is always executed when the class is being initiated and a new object is created.
- It is used to initialize object data

```
class Person:
    species = "Human" # Class attribute

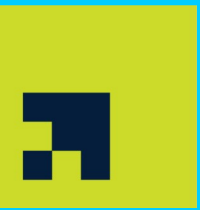
    def __init__(self, name, age): #
        self.name = name # Instance attribute
        self.age = age # Instance attribute
```



Attributes:

- name
- age

- **`self`** refers to the **current object**, allowing each object to store and access its own data
- **`self.name`** and **`self.brithday`** are instance attributes, unique to each Person object created from the class



The `__init__()` Method

- Using `__init__()` makes it easier to create objects with initial values:

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
p1 = Person("Alex", 28)
```

```
print(p1.name)
```

```
print(p1.age)
```




Default Values in `__init__()`

[Hunt 2019]

14

- We can also set default values for parameters in the `__init__()` method:

```
class Person:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age
```

```
p1 = Person("Emil")
p2 = Person("Tobias", 25)
```

```
print(p1.name, p1.age)
print(p2.name, p2.age)
```



Multiple Parameters

[Hunt 2019]

15

```
class Person:
    def __init__(self, name, age, city, country):
        self.name = name
        self.age = age
        self.city = city
        self.country = country

p1 = Person("Linus", 30, "Oslo", "Norway")

print(p1.name)
print(p1.age)
print(p1.city)
print(p1.country)
```



Calling Methods

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return "Hello, " + self.name

p1 = Person("Tobias")
p1.welcome()
```



Calling Methods with *self*

- We can also call other methods within the class using *self*:

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return "Hello, " + self.name

    def welcome(self):
        message = self.greet()
        print(message + "! Welcome to our website.")

p1 = Person("Tobias")
p1.welcome()
```



Class Properties

- We can also call other methods within the class using *self*:

Access Properties:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("Emil", 36)
```

```
print(p1.name)
print(p1.age)
```

Modify Properties:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("Tobias", 25)
print(p1.age)
```

```
p1.age = 26
print(p1.age)
```



Delete Properties

- We can delete properties from objects using the **del** keyword:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Linus", 30)

del p1.age

print(p1.name) # This works
# print(p1.age) # This would cause an error
```



Class Properties vs Object Properties

- Properties defined inside `__init__()` belong to each object (instance properties).
- Properties defined outside methods belong to the class itself (class properties) and are shared by all objects:

```
class Person:
    species = "Human" # Class property

    def __init__(self, name):
        self.name = name # Instance property

p1 = Person("Emil")
p2 = Person("Tobias")

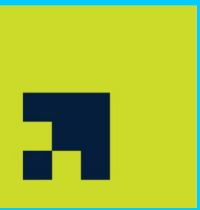
print(p1.name)
print(p2.name)
print(p1.species)
print(p2.species)
```




Providing a Class Comment

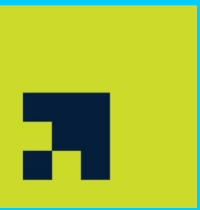
- It is common to provide a comment for a class
 - Defining what that class does, its purpose, and any important points to note about the class
 - This can be done by providing a docstring for the class just after the class declaration header using triple quotes string (" " "..." " " ")
 - The docstring is accessible through the `__doc__` attribute of the class
 - The intention is to make information available to users of the class, even at runtime
 - It can also be used by IDEs to provide information on a class

```
class Person:
    """
    An example class to hold a
    person's name and age.
    """
    ...
```



Automatic Memory Management

- Python Memory Manager provides automatic memory management
 - It allocates a portion of memory as and when required
 - When memory is short, it looks for areas which are no longer referenced
 - These areas of memory are then freed up (deallocated) so that they can be reallocated
 - This process is often referred to as Garbage Collection
- The provision of a memory manager (also known as automatic memory management) is one of Python's advantages
 - When compared to languages such as C and C++



Intrinsic Attributes

- Every class/object in Python has a set of intrinsic attributes set up by the Python runtime system
 - All start and end with a double underbar
 - Attributes/Methods that start with a double underbar ('__') are by convention considered special in Python
 - For classes
 - `__name__`
 - The name of the class
 - `__module__`
 - The module (or library) from which it was loaded
 - `__bases__`
 - A collection of its base classes (see inheritance)
 - `__dict__`
 - A dictionary (a set of key-value pairs) containing all the attributes (including methods)
 - `__doc__`
 - The documentation string
 - For objects
 - `__class__`
 - The name of the class of the object
 - `__dict__`
 - A dictionary containing all the object's attributes



Class Methods

- Methods are functions that belong to a class
- They define the behavior of objects created from the class

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello, my name is " + self.name)

p1 = Person("Emil")
p1.greet()
```

- All methods must have *self* as the first parameter



Methods with Parameters

- Methods can accept parameters just like regular functions:

```
class Calculator:
    def add(self, a, b):
        return a + b

    def multiply(self, a, b):
        return a * b

calc = Calculator()
print(calc.add(5, 3))
print(calc.multiply(4, 7))
```



Methods Accessing Properties

- Methods can access and modify object properties using *self*:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def get_info(self):
        return f"{self.name} is {self.age} years old"

p1 = Person("Tobias", 28)
print(p1.get_info())
```



Methods Modifying Properties

- Methods can modify the properties of an object:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def celebrate_birthday(self):
        self.age += 1
        print("Happy birthday! You are
now", self.age)

p1 = Person("Linus", 25)
p1.celebrate_birthday()
p1.celebrate_birthday()
```



The `__str__()` Method

[Hunt 2019]

28

- The `__str__()` method is a special method that controls what is returned when the object is printed:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("Emil", 36)
print(p1)
```

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} ({self.age})"
```

```
p1 = Person("Tobias", 36)
print(p1)
```




Delete Methods

- We can delete methods from a class using the *del* keyword:

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello!")

p1 = Person("Emil")

del Person.greet

p1.greet() # This will cause an error
```



Class-Side Methods

[Hunt 2019]

30

- It is also possible to define behaviour that is linked to the class rather than an individual object
 - This behaviour is defined in a class-side method
 - Class-side methods are written in a similar manner to any other method but are decorated with `@classmethod` and take a first parameter
 - Which represents the class rather than an individual instance

```
class Person:

    population = 0

    @classmethod
    def increment_population(cls):
        cls.population += 1

    def __init__(self, name, age):
        Person.increment_population()
        self.name = name
        self.age = age
```



Static Methods

- Static methods are defined within a class but are not tied to either the class nor any instance of the class
 - They do not receive the special first parameter representing either the class (*cls* for class methods) or the instances (*self* for instance methods)
 - They are in effect the same as free standing functions but are defined within a class
 - Often for convenience or to provide a way to group such functions together
 - A static method is a method that is decorated with the `@staticmethod` decorator
 - Both Java and C# uses the terms class-side methods and static methods interchangeably
 - Both those methods are the equivalent of class-side methods in Python
 - In Python, class-side methods and static methods are two very, very different things

```
class Person:
    @staticmethod
    def static_function():
        print("Static method")

Person.static_function()
```



Four Pillars of OOP

- The Four Pillars of Object-Oriented Programming (OOP) form the foundation for designing structured, reusable, and maintainable software:
 1. Inheritance
 2. Polymorphism
 3. Encapsulation
 4. Data Abstraction



Inheritance

- Inheritance allows a class (child class) to acquire properties and methods of another class (parent class).
- It supports hierarchical classification and promotes code reuse.

```
class Animal:

    # attribute and method of the parent class
    name = ""

    def eat(self):
        print("I can eat")

# inherit from Animal
class Dog(Animal):

    # new method in subclass
    def display(self):
        # access name attribute of superclass using self
        print("My name is ", self.name)

# create an object of the subclass
labrador = Dog()

# access superclass attribute and method
labrador.name = "Rohu"
labrador.eat()
```



Polymorphism

[Hunt 2019]

34

- Polymorphism means *"same operation, different behavior"*
- It allows functions or methods with the same name to work differently depending on the type of object they are acting upon

```
class Bird:  
    def sound(self):  
        return "Tweet"
```

```
class Dog:  
    def sound(self):  
        return "Bark"
```

```
def make_sound(animal):  
    print(animal.sound())
```

```
# Create objects
```

```
bird = Bird()
```

```
dog = Dog()
```

```
# Same function call, different behaviors
```

```
make_sound(bird) # Output: Tweet
```

```
make_sound(dog) # Output: Bark
```



Encapsulation

- Encapsulation is about protecting data inside a class
- It means keeping data (properties) and methods together in a class, while controlling how the data can be accessed from outside the class
- This prevents accidental changes to your data and hides the internal details of how your class works

```
class Tree:  
    def __init__(self, height):  
        self.__height = height
```

```
pine = Tree(20)  
pine.__height
```

AttributeError: 'Tree' object has no attribute '__height'



Encapsulation

- We can access or modify data by defining get() or set() method:

```
class Tree:
    def __init__(self, height):
        self.__height = height

    def get_height(self):
        return self.__height
```

```
pine = Tree(20)
pine.get_height()
```

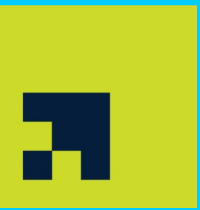
```
class Tree:
    def __init__(self, height):
        self.__height = height

    def get_height(self):
        return self.__height

    def set_height(self, new_height):
        self.__height = new_height
```

```
pine = Tree(20)
print(pine.get_height())
```

```
pine.set_height(10)
print(pine.get_height())
```

Why Use Encapsulation?

- Encapsulation provides several benefits:
 - *Data Protection*: Prevents accidental modification of data
 - *Validation*: You can validate data before setting it
 - *Flexibility*: Internal implementation can change without affecting external code
 - *Control*: You have full control over how data is accessed and modified



Data Abstraction

- Data abstraction is the process of hiding unnecessary and complex implementation details from users while showing only essential details and functionalities
- An abstract class in Python is a class that contains one or more abstract methods, methods that are declared but not implemented
- It acts like a blueprint for other classes
- We can't create objects directly from an abstract class. Instead, we create objects from its subclasses, which must implement the abstract methods



Data Abstraction

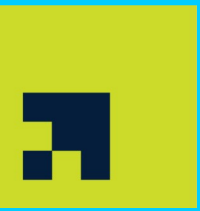
- We use the *abc module* to define abstract classes
- We create an abstract class `Animal` with an abstract method `sound()`, then implement it in the `Dog` class to return "Bark"

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):  
    @abstractmethod  
    def sound(self):  
        pass
```

```
class Dog(Animal):  
    def sound(self):  
        return "Bark"
```

```
# Creating an object of the Dog class  
dog = Dog()  
print(dog.sound())
```



Why Use Abstract Classes?

- To set a common structure for all subclasses
- They define structure but leave implementation to derived classes
- They help maintain consistency in code